

Deploying Lightweight Input-Aware Selective Instruction Duplication in HPC Applications

Md Hasanur Rahman
University of Florida
Gainesville, USA
mrahman7@ufl.edu

Guanpeng Li*
University of Florida
Gainesville, USA
liguanpeng@ufl.edu

Abstract

Modern high-performance computing (HPC) applications are increasingly vulnerable to silent data corruptions (SDCs) caused by transient hardware faults. While selective instruction duplication (SID) offers an efficient software-level protection strategy, existing SID methods rely on SDC vulnerability profile derived from only the default reference input often found in application suites. However, they overlook the input-dependent nature of SDC propagation. This leads to significant SDC coverage loss when inputs vary. We present PROTEGO, a novel input-aware SID protection framework that efficiently adapts protection to runtime inputs. PROTEGO performs a one-time vulnerability-guided input exploration to identify a small number of input groups with distinct SID protection patterns. At runtime, PROTEGO uses lightweight features derived from input arguments to select and deploy the appropriate SID protection. Our evaluation across 10 HPC applications demonstrates the effectiveness and efficiency of PROTEGO in mitigating SDC coverage loss across diverse inputs, compared to existing SID techniques.

CCS Concepts

• Computing Systems → High-performance Computing; • Dependable System Design → Soft Error Mitigation.

Keywords

High-performance Computing, Soft Errors, Silent Data Corruption, Selective Instruction Duplication, Lightweight Protection

ACM Reference Format:

Md Hasanur Rahman and Guanpeng Li. 2025. Deploying Lightweight Input-Aware Selective Instruction Duplication in HPC Applications. In *The International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*, November 16–21, 2025, St Louis, MO, USA. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3712285.3759822>

1 Introduction

As HPC systems scale in complexity and transistor dimensions shrink, they become increasingly susceptible to transient hardware

faults [8, 15, 33, 36]. These faults, often caused by radiation-induced bit flips or electrical noise, can lead to *silent data corruptions* (SDCs), where the system produces incorrect results without any indication of failure. SDCs are particularly concerning in scientific computing, where the correctness of application outputs is critical. Without effective detection, SDCs may silently invalidate simulation outcomes or data-driven discoveries, compromising the integrity of the simulation of large-scale HPC applications [3, 34, 35].

To mitigate SDCs, both hardware and software-based protection techniques have been explored [13, 24, 27, 32, 42]. At the hardware level, methods such as triple modular redundancy (TMR) [24] and circuit hardening [42] can mask or detect faults. However, these techniques incur significant area, power, and energy costs, limiting their scalability in HPC environments. As a more lightweight alternative, software-level techniques have gained traction [27, 32]. One such straightforward approach is *full duplication* [27], which duplicates all regions in the application irrespective of their SDC vulnerability and check the results at runtime to detect the error. However, this is very expensive because of redundant execution.

To resolve this, researchers have proposed *Selective Instruction Duplication* (SID) [18, 21, 23] based on the observation that only a small portion of instructions in an application are responsible for almost all the SDCs in an application [9]. Thus, protecting the most vulnerable parts of the application may be sufficient to achieve high SDC coverage with relatively low overhead. SID relies on fault injection (FI) experiments to evaluate each instruction's contribution to total SDC coverage. Then, it uses a cost-benefit optimization, modeled as a knapsack problem, to select which instructions to protect under a given protection overhead constraint.

Despite its efficiency, existing SID protection methods [9, 18, 23] decide the selection of instructions based on FI analysis under a default reference input often available in application benchmark suite. They assume that the SDC vulnerability profile remains consistent across inputs. However, in practice, as shown in prior works [31, 40], HPC applications exhibit highly input-sensitive behaviors – different inputs can activate different control paths and data regions, leading to shifts in SDC propagation patterns. As a result, existing SID protection plan derived using the default reference input may underprotect critical regions or overprotect benign ones under different inputs. This leads to significant SDC coverage loss, which is defined as the reduction in the percentage of detected SDCs when a protection plan derived from one input is applied to another.

To address the above limitation, we propose PROTEGO, a lightweight input-aware SID protection technique that adapts protection strategies to runtime inputs without requiring expensive FI or dynamic analysis. To the best of our knowledge, this is the first attempt to develop such an efficient input-aware SID protection technique to

*Corresponding author: Guanpeng Li.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SC '25, St Louis, MO, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1466-5/25/11
<https://doi.org/10.1145/3712285.3759822>

deploy at runtime. However, developing such technique is challenging in practice. First, the input space of modern HPC application is often massive, so evaluating SID protection patterns for each new input is impractical. Second, at runtime, we cannot perform expensive FI campaigns under given input to deploy the SID variant. Third, to develop such a lightweight runtime solution, we also cannot perform dynamic analysis of the application to assess the SDC propagation shifts as this would be comparable to deploying full duplication.

To address the above challenges, our key insights are as follows: (1) Although application input space is huge, the number of SID protection variants across inputs are only a few, so we can tailor to the appropriate SID protection for each such case, and (2) different transformations on the argument values of the runtime input can be served a proxy to the dynamic analysis of the application. Based on the above observations, PROTEGO develops an efficient input-aware SID protection technique that leverages SID-variant information learned from the one-time offline input group formation.

Our contributions are as follows.

- We perform a one-time FI-guided vulnerability search for exploring meaningful inputs that can maximally capture whole spectrum of SID protection variants.
- Based on our one-time search analysis, we form a handful of input groups in terms of SID protection variance.
- We develop and evaluate PROTEGO, a runtime system that selects accurate SID protection plans – without expensive runtime FI or profiling – by approximating application SDC propagation behavior.
- Our experimental results show that PROTEGO reduces the average SDC coverage loss from 21% to 10% across diverse inputs, compared to existing SID methods, over 10 widely used HPC applications.
- A case study demonstrates that PROTEGO is particularly effective in mitigating SDC coverage loss when encountered with high SDC-prone inputs. In this scenario, existing SID techniques would always fail to provide the expected SDC coverage. In practice, such inputs particularly cause major shifts in SDC patterns, rendering existing static SID approaches unable to meet the reliability targets during deployment.

2 Research Background

In this section, we present the definition and existing deployment steps of selective instruction duplication. Additionally, we discuss the fault injection methodology adopted in this study.

2.1 Selective Instruction Duplication (SID)

Selective Instruction Duplication (SID) is a widely adopted software-level protection technique designed to mitigate SDCs resulting from transient hardware faults [9, 18, 23]. Alternative approaches such as full duplication [27] or triple modular redundancy [24] may offer full SDC coverage, but they impose substantial performance overhead because of redundant computing/resource usage.

Figure 1 presents a comparison between full duplication and selective instruction duplication techniques. In the figure, each circle denotes an instruction, where light green circles indicate

original instructions and light orange circles represent their duplicated counterparts. As illustrated in Figure 1(b), full duplication indiscriminately duplicates all instructions, regardless of their SDC vulnerability characteristics, and inserts a checker before each synchronization point, such as a store or comparison instruction. At runtime, SDC is reported if any of the checkers detects any mismatch. Consequently, full duplication incurs significant overhead in real-world HPC environments, where applications may consist of millions of instructions.

On the other hand, Figure 1(c) illustrates that, under a given protection overhead constraint, SID strategically duplicates only a subset of critical instructions that are most likely to cause SDCs, without exceeding the allowed overhead. This is based on observation from prior studies [9, 23, 31] that a small subset of instructions in an application is typically responsible for the majority of SDCs. In this specific example, only the instructions labeled H and J are duplicated. This selective approach enables fine-grained control over protection overhead while still achieving high SDC coverage.

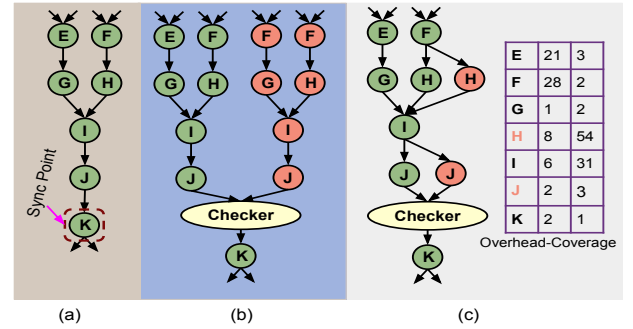


Figure 1: Illustrating the Example of Different Types of Protection Techniques. (a) Unprotected Code Segment. (b) Full Duplication. (c) Selective Instruction Duplication at 10% Protection Overhead.

2.2 Deployment Steps of Existing SID Technique

Given an application, existing SID protection methods [9, 18, 23] operate by evaluating the SDC probability of each instruction under the default reference input. This default reference input is typically provided with the application benchmark suite and is commonly used to analyze the application for software testing purposes.

To evaluate instruction-level SDC probability under an input, existing method first conducts fault injection (FI) campaigns. Note that we discuss our FI methodology in details in the next subsection. After evaluating instruction-level SDC probabilities from FI results, the protection overhead of each instruction is also measured. Under an input, the protection overhead of an instruction is typically computed by profiling the dynamic execution frequency of that instruction relative to the application's total execution cycles.

To determine which instructions to duplicate, SID formulates the protection strategy as a classic 0–1 knapsack optimization problem [6]. Each instruction is associated with a cost (second column of the Table in Figure 1(c)), reflecting its protection overhead when duplicated, and a benefit (third column of the Table), reflecting the expected reduction in overall SDCs if it is duplicated, which is also known as SDC coverage. Once these values are determined from

dynamic profiling and FI campaigns, the knapsack optimization selects the subset of instructions that maximizes SDC coverage given a user-defined protection budget.

We illustrate this selection process with an example overhead vs coverage results of 7 instructions in tabular form in Figure 1(c). In the example, the protection budget is set to 10%. Given the protection budget, the instructions selected by the optimization algorithm are H & J. These instructions are optimal within this protection budget because maximum possible SDC coverage is 57% at protection overhead 10%. Although it is possible to select a different set of instructions such as G, I, & J, but those would provide total SDC coverage of 36%, which is significantly lower at that protection overhead. After the selection process is completed, the selected instructions are then duplicated at compile-time to ensure protection during runtime execution.

2.3 Fault Injection (FI) Methodology

Existing SID techniques typically rely on FI campaigns to evaluate both overall and instruction-level SDC probabilities of an application [18, 22, 23]. In this study, we adopt a well-established fault model widely used in recent resilience studies [9, 12, 17, 22, 31, 39–41] to conduct our FI campaigns. This model focuses on faults occurring within processor’s computational components, such as pipeline stages, flip-flops, and functional units. It does not account for faults in memory, caches, instruction encoding, or control logic. The rationale behind this exclusion is that memory errors are generally mitigated by industry-standard error-correcting codes (ECC) [10, 38], while faults in instruction encoding or control logic are assumed to be detectable by existing control-flow checking techniques [11, 26].

Our fault model specifically focuses on datapath faults that manifest during application execution. Such faults may result in one of the following consequences: (1) the fault may be masked during error propagation, (2) it may lead to the application crash or hang, or (3) it may propagate and ultimately affect the application output. When a fault alters the application output, we classify it as a silent data corruption (SDC). SDC is the most severe of the three as it silently corrupts the application output without any noticeable symptoms. We define SDC probability as the likelihood that a fault results in a SDC.

We follow the same FI methodology, which is widely adopted in many resilience studies [10, 12, 17, 22, 31, 39–41]. Specifically, to evaluate the overall application SDC probability under an input, we conduct 1000 random faults under that input. Also, to derive instruction-level SDC probability under an input, we inject 100 random faults to each instruction under that input. Our FI measurement yields an error bar from 0.30% to 3.23% for the 95% confidence intervals, which is consistent with prior works [9, 10, 19, 20, 28–30, 39, 40].

For each FI trial, our fault model first randomly selects an instruction at the LLVM IR level. It then randomly chooses a bit position in the destination register of the selected instruction. At the designated bit location, the application execution is temporarily halted, and the bit value is flipped before resuming execution. The application is then allowed to run to completion, and its output is compared against the fault-free output. If any discrepancy is detected, it is classified as a silent data corruption (SDC).

We conduct our FI campaigns at LLVM IR level due to its practical and widely adopted features. LLVM provides a typed intermediate representation (IR) that retains source-level semantics, enabling precise error analysis. Its platform-neutral design abstracts hardware details, simplifying portability and low-level handling. LLVM has also been proven effective abstraction layer for fault injection studies [37, 41], and is extensively used in resilience research [1, 2, 9, 17, 21, 28, 29, 31, 41].

3 Motivation

In this section, we present the motivation for developing a runtime-efficient, input-aware SID protection technique. We begin by discussing the limitations of existing SID protection methods and conclude by outlining the key research challenges that must be addressed to overcome these limitations.

3.1 Limitation of Existing SID Techniques

Despite the effectiveness of SID protection in reducing SDCs with manageable overhead, existing methods [18, 22, 23] suffer from a critical limitation: they are largely input-agnostic and rely solely on the SDC vulnerability profile generated using the default reference input. These methods operate under the assumption that SDC propagation behavior remains stable across different application inputs. As a result, a one-time FI experiment is used to evaluate instruction-level SDC vulnerabilities. Based on this profile, SID protection is applied. The same statically protected executable is then deployed universally, regardless of the actual input used at runtime.

However, real-world applications often exhibit highly input-dependent behavior, including variations in control flow, data dependencies, and memory access patterns. These variations can cause significant shifts in instruction execution frequencies and SDC propagation characteristics. For example, instructions identified as critical under the default reference input may become inactive under a different input, while new vulnerable regions may become active and remain unprotected. As a result, when executed with a new input, existing static SID techniques may overprotect non-critical paths and underprotect truly critical ones. This mismatch leads to suboptimal SDC coverage under a fixed protection overhead budget.

The root of this limitation lies in SID’s reliance on cost-benefit optimization, which is highly sensitive to changes in SDC propagation behavior. Under a given protection overhead budget, the optimization ranks instructions based on their SDC probabilities under a specific input. When the SDC propagation pattern shifts due to a different input, the cost-benefit optimization outcome also changes. However, since instruction selection is fixed at compile time in existing SID techniques, the resulting static SID configuration no longer aligns with the actual SDC distribution under the new input. This misalignment leads to a noticeable drop in actual SDC coverage during deployment.

We illustrate this limitation in Figure 2 using two HPC applications: *Pathfinder* and *Particlefilter* with a subset of instructions. The Venn diagrams compare the instructions selected by the existing static SID technique (purple) with those selected (green) based on FI results under a different input *Inp*, assuming a 10% protection

overhead budget. The yellow region highlights the commonly selected instruction sets between the two methods. As shown, the two sets of selected instructions differ significantly for both applications. Due to this disparity, the existing SID technique suffers from substantial SDC coverage loss when deployed with input *Inp*: from an expected 43% to 29% for *Pathfinder*, and from 51% to 36% for *Particlefilter*. Note that we introduce the applications in details in Section 5.2.

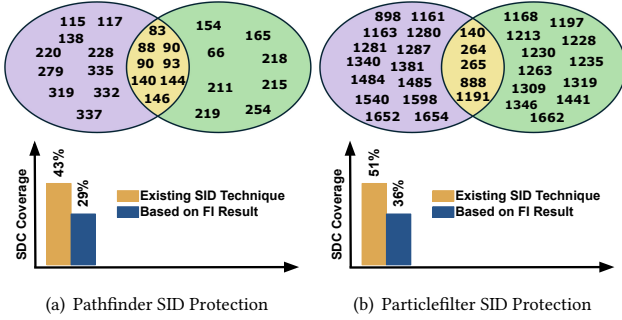


Figure 2: Illustrating the Limitations in Existing SID Techniques by Highlighting the Change in Instruction Selection for Duplication.

To further quantify this limitation, we conduct an experiment using 30 randomly selected inputs for each of 10 HPC applications. For each input of an application, we deploy the static SID plan generated by existing SID method and measure the resulting SDC coverage at two protection levels: 10% and 30%. The results are presented in Figure 3, where each bar represents the observed range of SDC coverage across the 30 inputs, and the green line indicates the SDC coverage offered by existing SID method under the default reference input.

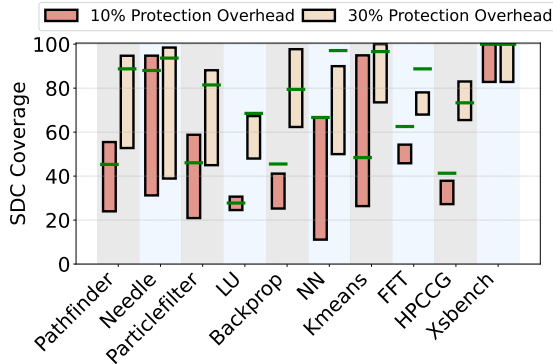


Figure 3: Demonstrating SDC Coverage Loss across Inputs When Using Reference Input-based SID Protection at 10% and 30% Protection Levels.

The Figure reveals that the actual SDC coverage across those 30 random inputs often falls significantly below the offered coverage by existing SID method for most applications. Specifically, we see that the lower bound of the coverage frequently dips well below the green reference line for most of the applications. These findings

confirm that static SID plans fail to consistently meet expected protection goals across diverse inputs, resulting in resilience levels that fall short of practical reliability targets.

Key Takeaway: Our observations confirm that existing SID protection techniques do not generalize across input variations, exposing the application to underprotection and increased risk of SDCs in deployment environments.

The above observations motivate our work on developing a lightweight runtime SID protection technique that accounts for potential variations in SDC behavior when encountering new inputs at runtime.

3.2 Challenges to Deploy Input-Aware Runtime Protection Technique

Designing an input-aware SID strategy that adapts to diverse SDC propagation behaviors is challenging due to the vast input space. Real-world applications may have millions of possible inputs, each potentially activating different control paths and instruction subsets. Exhaustively enumerating or analyzing all inputs is computationally infeasible. This input diversity implies that a runtime input-aware SID strategy would need to dynamically re-evaluate SDC vulnerabilities and update protection decisions for each new input. However, such dynamic analysis is not only impractical during deployment but also contradicts the original design goal of minimizing runtime overhead and avoiding redundant execution.

The most straightforward approach to evaluate SID under each input would be to perform FI campaigns tailored to that specific input and update the instruction selection accordingly. However, this method is prohibitively expensive. Evaluating SID requires injecting faults into thousands of instruction instances and monitoring the resulting application behavior, which demands substantial time and computational resources. In fact, conducting such detailed FI for every instruction under each input would incur a higher overhead than simply executing the application with full duplication, thereby defeating the purpose of using SID as a lightweight protection technique.

Alternatively, one might consider collecting dynamic execution information – such as the set of exercised instructions and their execution frequencies – at runtime to adjust the SID plan. However, this form of dynamic analysis introduces significant computing overhead, often comparable to that of full duplication. Tracking every dynamic instruction and evaluating its cost-benefit characteristics online requires intrusive instrumentation and incurs synchronization and decision-making costs that can substantially degrade performance. Moreover, determining when to stop such profiling during execution adds further complexity and unpredictability. These limitations render the approach impractical, particularly for time-sensitive or large-scale HPC applications.

Key Takeaway: Designing an efficient runtime input-aware SID protection strategy is fundamentally challenging due to the massive input space and the cost of re-evaluating SDC vulnerabilities per input. Both FI-based evaluation and dynamic profiling incur overheads comparable to or exceeding full duplication, hindering their worthiness.

4 PROTEGO: Lightweight Input-Aware SID Protection Technique

In this section, we introduce PROTEGO, a lightweight, input-aware runtime SID protection framework.

4.1 Overall Design Overview

The overarching goal of our PROTEGO is to enable efficient and input-aware SID protection by dynamically selecting the most suitable protection strategy at runtime. Based on our observations, existing SID protection approaches [18, 22, 23] typically rely on static instruction selection strategies, which do not generalize well across diverse inputs. As discussed in Section 3.1, variations in input can lead to significant shifts in optimal protection strategies. To overcome this limitation, PROTEGO introduces a systematic one-time offline analysis phase, followed by the deployment of a lightweight runtime protection method that maintains high SDC coverage with minimal overhead.

Figure 4 shows the overall workflow of PROTEGO. During the offline phase, PROTEGO performs a one-time SDC vulnerability-guided search across a diverse set of application inputs that exhibit significant variation in SDC behavior. This analysis characterizes how SID protection strategies differ across inputs. Based on the observed variations, PROTEGO partitions the input space into a small number of input groups, where inputs within each group share similar protection patterns. For each input group, PROTEGO statically generates and stores a SID-protected executable – denoted as representative executable – that represents the SID strategy best suited for that group.

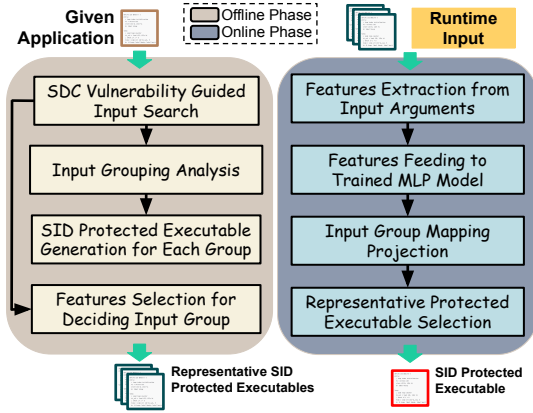


Figure 4: Overall Design Workflow of PROTEGO

At runtime, when a new input is encountered, PROTEGO efficiently extracts lightweight and highly informative features from the given input argument values using preselected argument transformations derived during the offline analysis. These features are then fed into a lightweight and highly accurate Multi-Layer Perceptron (MLP) model to predict the most appropriate input group. The MLP model is trained during the offline phase using the information based on the inputs explored by our search strategy. Based on the predicted input group, the representative SID protected executable stored for that group is returned to the user, enabling adaptive and accurate SID protection tailored to the runtime input.

This design ensures that PROTEGO avoids expensive FI campaigns or dynamic analysis at runtime, yet maintains high accuracy in selecting the correct SID protection variant, resulting in a practical, scalable, and input-sensitive SDC mitigation solution for HPC applications.

4.2 Design Details of Key Components

Here we present the design details of the key components of our technique PROTEGO.

4.2.1 A One-time SDC Vulnerability Guided Search. To build an efficient input-aware SID protection strategy at runtime, it is essential to explore a small yet informative set of inputs that capture the diversity of SDC vulnerability patterns across the application’s input space. PROTEGO addresses this by performing a one-time guided input search aimed at uncovering inputs that trigger significantly different SDC propagation behaviors. This guided exploration is critical because SID protection patterns are highly sensitive to how SDC vulnerabilities shift across different regions of an application under varying input data. As a result, just randomly exploring the input space would fail to efficiently capture the full spectrum of SDC vulnerabilities within a reasonable time frame.

To efficiently discover these informative inputs, PROTEGO employs an optimization-based search approach that iteratively evolves input candidates with the goal of maximizing diversity in SDC vulnerability behavior. The search begins with a small set of seed inputs. In each iteration, new inputs are generated by applying mutation or crossover operations to the current input pool. For mutation, a random input from the pool is selected, one of its argument indices is randomly chosen, and finally the value at that index is slightly perturbed. For crossover, two random inputs from the pool are selected, a random argument index is chosen, and the values at that argument index are exchanged between the two inputs. Overall, these operations enable the search to explore new regions of the input space while maintaining a controlled and directed convergence path.

The search terminates after discovering 500 unique inputs. This number is selected to capture a wide range of SDC vulnerability profiles while keeping the FI campaign computationally tractable. In practice, HPC applications typically demonstrate extremely large and diverse input spaces [25, 31], making exhaustive exploration with FI campaigns infeasible. Instead, our design strategically selects a subset of 500 inputs that captures meaningful variability in SDC behavior. As shown in our evaluation, PROTEGO achieves high effectiveness based on this carefully explored pool of 500 inputs, demonstrating that reliable SDC coverage insights can be obtained even from this very small fraction of the total input space.

During the search, each newly generated input is evaluated through FI campaigns to assess the SDC vulnerabilities of different regions of the application under that input. After the SDC evaluation, PROTEGO assigns a candidate score to the input based on how much its vulnerability profile deviates from those already observed. Inputs with higher deviation are prioritized, as they are more likely to reveal distinct SID protection variants.

We illustrate the input prioritization with an example in Figure 5. In the Figure, each rectangular box associated with an input represents the SDC probability of the instruction at that index. We only

show the SDC data of few instructions for illustration purposes. We assume that inputs P and Q are already discovered in previous iterations, so newly generated inputs I and J at current iteration will be assigned a score when comparing its SDC profile of each instruction to the corresponding SDC profiles of inputs P and Q . We see that based on the comparison, input I is prioritized because it show a very different SDC profile compared to what the search already have. In contrast, input J gets lower priority.

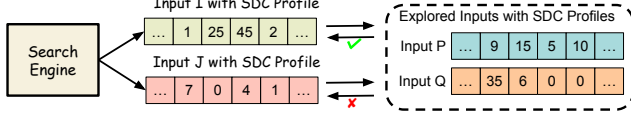


Figure 5: Illustrating Input Prioritization in Search.

This scoring function to prioritize inputs ensures that the search remains focused on uncovering meaningful diversity rather than redundant input behaviors. Note that the entire vulnerability-guided search is performed only once per application, making it a practical and scalable foundation for the runtime framework.

The SDC sensitivity information gathered from this one-time search serves as the backbone to deploy our runtime protection strategy with minimal overhead.

4.2.2 Input Grouping Analysis. After identifying a set of meaningful and informative inputs with diverse SDC vulnerability characteristics through the one-time guided search, the next step for PROTEGO is to organize these inputs into a small number of representative input groups based on the similarity in SID protection patterns. The motivation behind this input grouping is to capture the full spectrum of SID protection variants while minimizing the number of runtime decisions. A smaller set of input groups ensures that runtime group classification remains fast and efficient, without compromising protection accuracy.

To form these input groups, we conduct an empirical study for each target application. For each of the 500 inputs explored during the guided search, we have already measured instruction-level SDC vulnerability profiles. The grouping strategy is based on the idea of minimizing the maximum SDC coverage loss within a group. We define SDC coverage loss as the reduction in SDC coverage when a protection strategy optimized for one input is applied to a different input. By controlling this loss within each group, we ensure that the same SID variant can be safely and effectively shared among inputs with similar vulnerability characteristics.

Our experiment incrementally increases the threshold of maximum allowable SDC coverage loss within any group, ranging from 0% to 100%. At each threshold level, groups are formed by iteratively selecting a representative input from the set of ungrouped inputs and using its SID protection plan as a reference. All remaining ungrouped inputs are then evaluated against this reference. If the SDC coverage loss for an input falls below the current threshold, it is placed in the same group. This process continues until all inputs are assigned to a group. The resulting number of groups is plotted against the maximum allowable SDC coverage loss to analyze the trade-off between protection accuracy and grouping efficiency.

Figure 6 presents the results for thresholds ranging from 0% to 50%. The results reveal a sharp decline in the number of input

groups as the maximum allowable coverage loss increases. For example, at a 10% threshold, we consistently observe only a few input groups per application. Based on this observation, we select the 10% threshold as it strikes a practical balance – preserving most of the protection accuracy while significantly reducing runtime complexity. Notably, our experiments show that the average SDC coverage loss within each group at this threshold is substantially lower than the 10% upper bound. As a result, PROTEGO adopts the 10% threshold across all applications and finalizes the number of input groups accordingly. For each group, we select the input with the median overall SDC rate as the representative input.

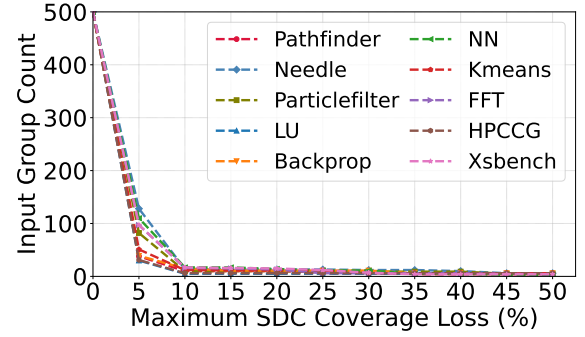


Figure 6: Illustrating The Existence of Small Number of Input Groups That Can Approximate Whole Spectrum of SID Variants.

After deciding on the representative input for each group, PROTEGO measures SID protection plan based on that representative input and store the corresponding protected executable. These small set of representative protected executables serve as a future reference for delivering a lightweight and accurate runtime SID protection solution when encountering a new unseen input.

4.2.3 Features Selection for Deciding Input Group. After generating the SID protected executable for each input group, the next step is to determine how to accurately and efficiently select the appropriate group for a new input at runtime. Achieving this next step faces several design challenges.

Design Challenges. A straightforward but impractical approach for determining the input group at runtime would be to perform FI campaigns during execution to measure the input's instruction-level SDC vulnerabilities and compare them against those of the existing groups. The group with the most similar SDC profile could then be selected. However, this method incurs prohibitive runtime cost and complexity, making it unsuitable for practical deployment.

Conducting FI campaigns at runtime is infeasible because each FI trial requires executing the entire application under fault conditions, which is more time-consuming than simply applying full duplication, where the application is run twice and outputs are compared. As discussed earlier, thousands of FI trials are typically needed to accurately evaluate SDC vulnerabilities with statistical confidence, making this approach even more expensive than full duplication-based protection. Since PROTEGO is designed to be significantly more lightweight than full duplication, relying on runtime FI-based input classification directly contradicts its core design objective.

Another alternative is to perform dynamic analysis to generate runtime execution trace and identify the quantifiable relationships between execution patterns and SDC vulnerabilities, which could then serve as input group indicators at runtime. However, this approach also incurs substantial overhead in the context of our problem. Dynamic tracing requires the application to execute under the input in question to collect profiling data, resulting in runtime costs comparable to those of full duplication. Consequently, dynamic analysis is also incompatible with our goal of lightweight and efficient SID protection.

Our Design Approach. To address the above challenges and effectively adapt to runtime input changes, PROTEGO must rely on informative knowledge that is (1) dynamic, (2) lightweight – requiring no runtime execution of the application – and (3) capable of capturing correlations with SDC vulnerability patterns.

To extract such informative knowledge, we observe that an application exercises different regions depending on the values of its input arguments. Furthermore, the frequency of execution for a given region can vary significantly across different inputs. This variation influences the SDC vulnerability ranking of the application, as regions executed more frequently are more likely to experience faults due to its random nature and thus exhibit higher probabilities of SDC vulnerability.

The above relationship is grounded in how SDC vulnerability is computed from FI results. Equation 1 presents the standard statistical formulation for calculating application SDC vulnerability, where $V_{application}$ denotes the overall SDC vulnerability/probability of an application, $Count_{SDC}$ is the number of FI trials that result in SDCs, and $Count_{total}$ is the total number of FI trials conducted. Equation 2 provides an equivalent formulation by introducing two components: V_i , the per-instruction SDC vulnerability/probability, and $Count_i$, the execution count of static instruction i during a dynamic run of the application. This formulation highlights that execution patterns, which are influenced by input arguments, directly affect the overall SDC vulnerability of an application.

$$V_{application} = Count_{SDC} / Count_{total} \quad (1)$$

$$V_{application} = \left(\sum_{i=1}^N V_i \cdot Count_i \right) / Count_{total} \quad (2)$$

$$= \sum_{i=1}^N Count_i \cdot (Count_i / Count_{total})$$

Based on the above formula, we also see that the overall SDC vulnerability under a given input can reflect shifts in SDC propagation across instruction space. As a result, overall SDC vulnerability of an application can serve as an indicator of any significant changes in instruction-level SDC vulnerabilities across inputs, which ultimately lead to variations in the corresponding SID protection plans.

We validate the above observation using Figure 7, which is based on FI results from two input groups: (1) a group of 30 low SDC-prone inputs and (2) a group of 30 high SDC-prone inputs, evaluated across 10 applications. For each input, we generate SID protection plans at a 10% overhead level using the corresponding FI results. We then measure the resulting SDC coverage for all inputs in both groups. To assess similarity within each group, we compute the

range of SDC coverage values separately for the low and high SDC-prone inputs. The expectation is that a narrow coverage range within each group indicates similar SID protection patterns. As shown in the figure, the range of SDC coverage is consistently low across all inputs in both groups. This result confirms that overall SDC vulnerability is a meaningful information for guiding input classification decisions.

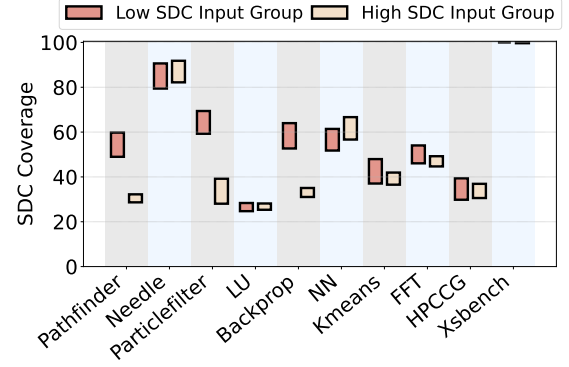


Figure 7: Illustrating the Impact of Application's SDC Propagation Behavior on Deciding Input Grouping at 10% Protection Overhead.

Key Takeaway: Input argument values can project the varying execution patterns in an application, which in turns helps approximate the application SDC vulnerability.

Based on our above observations, PROTEGO resorts to different argument-level features of the given input to project/estimate overall SDC vulnerability of the application under that input, which can be used later to select the appropriate input group. Note that extracting these features requires no FI or dynamic analysis. Upon comparing the estimated overall SDC vulnerability with the pre-determined (during offline analysis) overall SDC vulnerability for each input group, PROTEGO is able to correctly and efficiently decide on the appropriate group for the given input.

Input Argument-based Features Selection. Given an input, the next step for PROTEGO is to determine which input argument-based features can effectively approximate the overall application SDC vulnerability.

To achieve this, we extract the raw argument values from the input and apply various transformations to generate candidate features. For example, if the input includes arguments $arg1$ and $arg2$, the corresponding transformed features may include expressions such as $arg1^2$, $arg2^2$, $arg1 \times arg2$, and $arg1/arg2$ and their corresponding $\log$ transformations.

From the initial set of input features generated through the above transformations, we apply a feature importance analysis to identify the top 5 most predictive features correlated with the overall application SDC vulnerability. This analysis is performed using the dataset of 500 inputs explored during the offline search phase. While it is possible to include more features, increasing the number of features relative to the limited number of training samples can lead to underfitting during machine learning training. To select the top

5 features, we assign an importance score to each candidate based on an effective technique known as mutual information. Mutual information quantifies the dependency between each feature and the target variable, helping to identify those features most strongly associated with predicting overall SDC vulnerability.

After selecting the top 5 features, we train a Multi-Layer Perceptron (MLP) regression model. Note that in our study we find that without extensive tuning, simpler models are less consistent across applications to model these complex relationships. Therefore, we opt to choose MLP model in our problem context. Specifically, the model is trained on data extracted from the information based on 500 explored inputs during the offline search, where each input provides one training example: the top 5 pre-determined argument-level transformations are used as feature vectors, and the measured overall SDC vulnerability under that input serves as the target.

The MLP architecture consists of 5 hidden layers with dimensions (512, 256, 128, 64, 32) to capture complex input versus SDC vulnerability relationships, and applies the ReLU activation function in the final layer. We employ the Adam optimizer with a batch size of 8 and train the model for 10000 iterations. These design choices are consistent across 10 applications and lead to accurate SDC vulnerability estimation. We quantify the effectiveness of our MLP model later in Section 4.2.4.

4.2.4 Online Phase of PROTEGO. Recall that the overarching goal of PROTEGO is to accurately and efficiently provide a SID-protected executable tailored to the given input at runtime.

At runtime, given a new input I , PROTEGO extracts a feature vector of length 5, where each feature is corresponds to one of the top 5 pre-determined (during training) input argument transformations. This feature vector is then passed to the trained MLP model to estimate the overall SDC vulnerability under input I . The expectation is that the predicted SDC vulnerability closely approximates the ground truth SDC vulnerability obtained via FI campaigns for the same input I . In Figure 8, we demonstrate that the trained model achieves a low Mean Absolute Error (MAE) between predicted and ground truth SDC rates during model inference phase, thereby ensuring accurate input group classification.

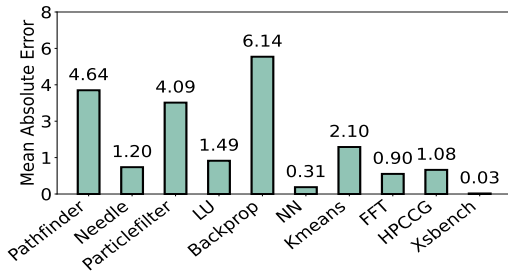


Figure 8: Mean Absolute Error Between Predicted Overall SDC Rate (by Our MLP Model) and True Overall SDC Rate Based across Online Inputs.

After estimating the overall SDC vulnerability under I , PROTEGO quickly look up the pre-determined overall SDC vulnerability under each representative input from each input group. The appropriate input group is then selected based on the closeness of the overall SDC vulnerabilities. Once the input group is determined, PROTEGO

Table 1: HPC Applications Used to Evaluate PROTEGO.

Application	Suite	Description
Pathfinder	Rodinia	Dynamic programming for shortest path in a grid
Needle	Rodinia	Global optimization for DNA sequence alignment
Particlefilter	Rodinia	Bayesian estimation of object location from noisy measurements
LU	Rodinia	Solves systems of linear equations
Backprop	Rodinia	Neural network training via backpropagation
NN	Rodinia	k-nearest neighbor search in unstructured data
Kmeans	Rodinia	Clustering algorithm used in data mining
FFT	SPLASH-2	1D fast Fourier transform using six-step method
HPCCG	Mantevo	Conjugate gradient solver for 3D chimney domain
Xsbench	CESAR	Monte Carlo neutronics kernel computation

quickly retrieves the associated representative SID protected executable for that group and returns it to the user.

5 Evaluation

In this section, we present the evaluation of our technique PROTEGO. We first demonstrate the effectiveness of PROTEGO in terms of mitigating SDC coverage loss compared to existing SID techniques, followed by the evaluation of execution time analysis for PROTEGO.

5.1 Experimental Setup

We conduct our experiments on a Linux server equipped with 24 processing cores and 512 GB of memory.

5.2 Evaluated HPC Applications

In our study, we evaluate PROTEGO using 10 real-world HPC applications drawn from 4 benchmark suites. These applications are widely used in recent resilience studies [1, 12, 14, 17, 21, 28, 31]. Table 1 presents our applications. Our application selection criteria are as follows: (1) applications should represent diverse computational patterns, including stencil, graph, numerical, and data-intensive workloads from various HPC domains; (2) application manuals must provide a clearly defined input format that the application expects; (3) the input format should support flexible and diverse random input generation to allow meaningful exploration of the input space; and (4) applications must be compilable to LLVM IR to enable FI campaigns at the instruction level.

5.3 The Mitigation of SDC Coverage Loss

Here we show the effectiveness of PROTEGO in mitigating SDC coverage loss that is observed when using the baseline method, which is existing SID technique under default reference input [9, 18, 23].

Recall that existing input-agnostic SID technique suffers from substantial SDC coverage loss when evaluated using different inputs. PROTEGO aims to mitigate this loss by proposing an efficient and input-aware runtime SID protection technique. Specifically, we first randomly choose 100 online inputs. Then, under each of those inputs, we conduct FI campaigns on protected codes derived from the baseline SID method and our technique PROTEGO at 10%, 30%, 50% and 70% protection overhead budgets. Note that for a fair evaluation, we make sure that these 100 online inputs are not explored during our one-time search stated in Section 4.2.1. After the FI campaigns on protected codes, we then observe the SDC coverage loss by baseline and PROTEGO under those 100 inputs.

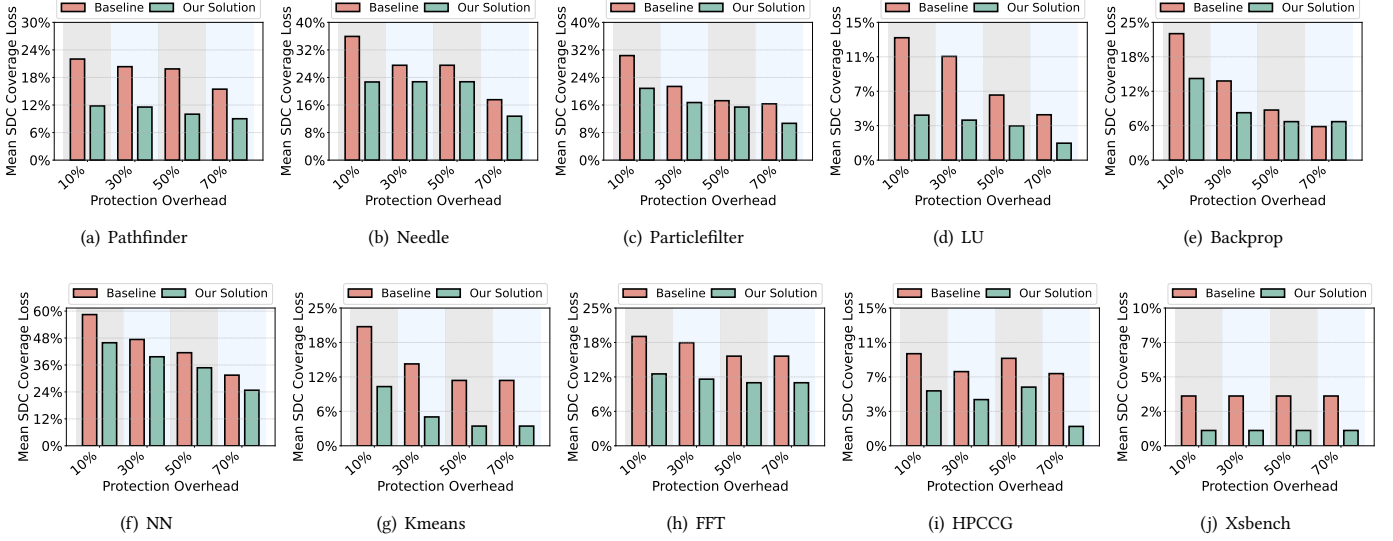


Figure 9: Demonstrating the Mitigation of Mean SDC Coverage Loss (in Percentage) across Inputs of An Application by PROTEGO When Compared to Existing SID method at 10%, 30%, 50% and 70% Protection Overheads.

Figure 9 demonstrates the results by comparing the mean SDC coverage loss, at different protection overheads, by the baseline and PROTEGO across those 100 inputs for each application. As shown in the Figure, our technique PROTEGO yields much lower mean SDC coverage loss than those by the baseline. For example, in LU application, the mean SDC coverage losses observed using the baseline at 10%, 30%, 50% and 70% protection overhead are 13.33%, 11.29%, 7.09% and 4.94% respectively, while they are only 4.88%, 4.35%, 3.73% and 1.84% respectively observed using our technique PROTEGO. On average across all 10 applications, the loss observed using baseline at 10%, 30%, 50% and 70% protection overhead are 23.81%, 18.76%, 16.36% and 13.14% respectively while they are only 15.18%, 12.71%, 11.62% and 8.43% respectively when using our technique PROTEGO.

Overall, the results confirms the effectiveness of PROTEGO in addressing the limitation of existing SID protection technique by significantly mitigating the SDC coverage loss.

5.4 Our Offline Analysis Time Breakdown

Recall that PROTEGO performs a one-time vulnerability guided search to decide on the input groups. Note that the search also includes the vulnerability evaluation time for each of the 500 explored inputs. For each explored input, we conduct parallel FI campaigns for faster evaluation. Later, FI campaigns are conducted to construct a representative SID variant for each group. Additionally, PROTEGO trains a MLP model to efficiently select appropriate input group at runtime and provide the user with the corresponding SID protection. Therefore, our offline analysis time is divided into three major steps: (1) input search time, (2) FI experiment and SID evaluation time for each representative input group, and (3) training time of MLP model.

Table 2 shows the breakdown results of our offline analysis time. On average across all 10 applications, search time takes around

Application	Pathfinder	Needle	Particlefilter	LU	Backprop
Input Search Time (Sec)	59079.00	349167.50	121132.00	312260.00	161996.00
SID Evaluation Time (Sec)	1525.00	62018.49	83948.27	13053.51	42949.20
Model Training Time (Sec)	3.74	2.94	3.98	7.91	5.29

Application	NN	Kmeans	FFT	HPCCG	Xsbench
Input Search Time (Sec)	221135.00	99253.75	210254.00	376271.43	281154.05
SID Evaluation Time (Sec)	14448.83	16758.98	17024.18	220160.18	61114.20
Model Training Time (Sec)	7.43	6.65	6.26	8.82	5.98

Table 2: Breakdown of Offline Analysis Time in Seconds.

3500 minutes, whereas representative input group SID evaluation takes around 880 minutes. Additionally, model training time only takes around 6 seconds. Note that all of these fall into just one-time analysis for each application. Based on the offline analysis, PROTEGO develop a very fast input-aware SID protection technique.

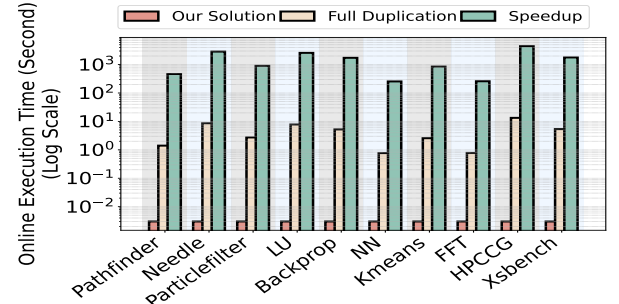


Figure 10: Execution Time Comparison (in Seconds) between Our Solution and Full Duplication, with Resulting Speedup Per Input of An Application. Data Is Shown in Log Scale for Better Readability.

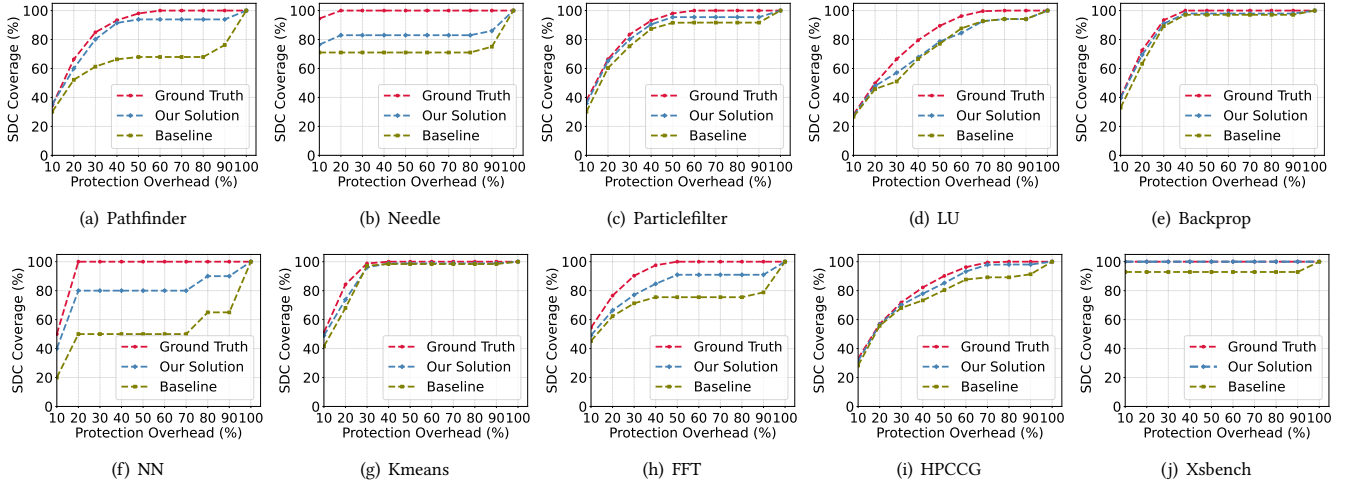


Figure 11: Demonstrating the Benefits of Using Our SID Technique PROTEGO Especially When Encountering Inputs Prone to High SDC Vulnerabilities.

Note that in our study, we observe that during the input search, inputs explored by later-stage of the search often exhibit similar SDC vulnerability profiles to the already explored ones. This suggests an opportunity to further reduce the offline cost by deciding when fault injection is necessary and when an existing SDC profile can be reused. We leave this as a potential future work.

5.5 Online Execution Time by PROTEGO

Recall that PROTEGO is an efficient input-aware SID protection technique that does not require any expensive FI campaigns or dynamic analysis at runtime. PROTEGO achieves that by promptly classifies the appropriate input group based on the given runtime input, which is followed by a very lightweight lookup at the hash table containing the representative SID-protected executables.

Figure 10 presents the runtime execution time to provide the users an appropriate SID protected executable. In the figure, the execution time by PROTEGO is compared with the time required by to deploy a full duplication [27]. The results are shown in log scale for better visibility. Specifically, on average across 10 applications, under a given online input, PROTEGO takes around 0.05 seconds to provide an appropriate SID-protected executable at runtime while full duplication may require around 5 seconds to deploy at runtime. Therefore, PROTEGO achieve a speedup of 100× compared to full duplication during runtime execution.

6 A Case Study with High SDC-prone Inputs

In this section, we conduct a case study to further illustrate the effectiveness of PROTEGO in mitigating SDC coverage loss.

We observe that existing SID technique especially experiences high SDC coverage loss in cases where application exhibit higher SDC vulnerability under an input. We call these inputs high SDC-prone inputs. The reason of high SDC coverage loss by existing SID technique is that high SDC-prone inputs often exercises application regions that are not explored much by default reference input used

in existing SID technique. As a result, those application regions may exhibit higher SDC vulnerabilities, which causes a significant shift in instruction selection process of SID under those high SDC-prone inputs, leading to high ranked instruction sets under default reference inputs shifted to low ranked. This drastic change in selected instructions results in significant SDC coverage loss with existing SID protection technique.

In Figure 11, we demonstrate the implication of the our above findings with the comparison of SDC coverage results among ground truth method (e.g., FI to evaluate the actual SDC coverage under the input), our technique PROTEGO, and existing SID technique. We measure SDC coverage by each technique at 10%, 20%, up to 100% protection overhead under one such high SDC-prone input for each application. As seen, for example, in NN application, compared to ground truth SDC coverage, existing SID technique experiences SDC coverage loss of around 85.76% when PROTEGO experiences only 16.00%. On average across all applications with such high SDC prone inputs, the percentage of SDC coverage loss by existing SID protection method is 22.89%, while it is only 6.41% for PROTEGO.

Protecting application considering high SDC-prone inputs is must needed because otherwise SID protection based on only default reference input may fail to meet reliability target in many practical in-situ scenarios. In such scenario, PROTEGO offers a significant advantage over existing SID technique because of its efficient input-aware SID protection technique.

7 Related Works

Instruction duplication has been extensively studied as a fault tolerance technique to mitigate transient hardware faults [9, 13, 23, 27]. In recent years, SID technique has gained popularity due to its offering a practical trade-off between protection coverage and performance overhead [9, 17, 18, 21, 23]. Instead of duplicating all

instructions, SID frameworks selectively protect a subset of instructions based on their vulnerability to SDCs. This approach significantly reduces protection execution overheads while maintaining high SDC coverage across many applications.

However, all of those works study resilience under a default reference input. Recent works have begun to recognize the limitations of static resilience studies when applied to varying application inputs. Multiple studies [7, 16, 25, 31, 40] have demonstrated that application vulnerability is input-sensitive. For instance, Yang et al. [40] explored GPU resilience by adjusting input sizing strategies while Mahmoud et al. [25] employed test case prioritization to enhance resiliency assessments. Rahman et al. [31] further confirmed that application overall vulnerability changes with input, and proposed methods to bound worst-case SDC behavior. However, these studies do not extend their findings into practical, input-aware SID technique that efficiently adapt the protection dynamically at runtime.

Moreover, fuzzing has emerged as a widely adopted tool for software testing and system validation [4, 5, 43]. While those works focus on mostly coverage-based search in finding software bugs, our work reimagines the fuzzing for input-aware resilience. Specifically, we propose a one-time, vulnerability-guided input search strategy driven by instruction-level SDC measurements. This approach enables the discovery of input-sensitive protection patterns and the formation of input groups with similar vulnerability profiles.

8 Conclusion

In this paper, we present PROTEGO, a lightweight and input-aware SID technique that mitigates the loss of SDC coverage observed using existing SID protection technique. Unlike existing SID methods that assume input-invariant vulnerability profiles, PROTEGO addresses the input sensitivity of SDC propagation by adapting SID protection plans based on given inputs without requiring fault injection or dynamic analysis at runtime.

PROTEGO achieves this by first performing a one-time, FI-guided search to uncover representative inputs that capture the diversity of SDC propagation behaviors across the input space. It then partitions the input space into a small number of groups based on SID protection similarity and pre-computes the representative SID protection for each group. At runtime, PROTEGO extracts the lightweight features from the input arguments and accurately maps the given input to appropriate input group, which is followed by promptly pulling up the corresponding representative SID-protected executable from that group. Therefore, PROTEGO enables fast and effective deployment with appropriate SID variant.

Our evaluation across a diverse set of 10 HPC applications demonstrates that PROTEGO significantly mitigates SDC coverage loss compared to existing input-agnostic SID techniques, especially under high-risk input scenarios. PROTEGO enables robust SID protection while preserving the low runtime overhead.

Acknowledgments

This work was supported by the U.S. Department of Energy (DoE), Office of Science, ASCR, under contract DE-SC0024559, as well as National Science Foundation (NSF) grant OAC-2410855 and 2441136.

References

- [1] Abdul Rehman Anwer, Guanpeng Li, Karthik Pattabiraman, Michael Sullivan, Timothy Tsai, and Siva Kumar Sastry Hari. 2020. GPU-trident: efficient modeling of error propagation in GPU programs. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15.
- [2] Rizwan A Ashraf, Roberto Gioiosa, Gokcen Kestor, Ronald F DeMara, Chen-Yong Cher, and Pradip Bose. 2015. Understanding the propagation of transient errors in HPC applications. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. ACM, 72.
- [3] Leonardo Bautista-Gomez, Ferad Zyulkyarov, Osman Unsal, and Simon McIntosh-Smith. 2016. Unprotected computing: A large-scale study of dram raw error rate on a supercomputer. In *SC'16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 645–655.
- [4] Marcel Böhme, Van-Thuan Pham, and Abhik Roychoudhury. 2016. Coverage-based greybox fuzzing as markov chain. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. 1032–1043.
- [5] Hongxu Chen, Yinxing Xue, Yuekang Li, Bihuan Chen, Xiaofei Xie, Xiuheng Wu, and Yang Liu. 2018. Hawkeye: Towards a desired directed grey-box fuzzer. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*. 2095–2108.
- [6] George B. Dantzig. 1957. Discrete-variable extremum problems. *Operations Research* 5, 2 (1957), 266–277. <https://doi.org/10.1287/opre.5.2.266>
- [7] Domenico Di Leo, Fatemeh Ayatollahi, Behrooz Sangchoolie, Johan Karlsson, and Roger Johansson. 2012. On the impact of hardware faults—an investigation of the relationship between workload inputs and failure mode distributions. In *International Conference on Computer Safety, Reliability, and Security*. Springer, 198–209.
- [8] Harish Dattatraya Dixit, Sneha Pendharkar, Matt Beadon, Chris Mason, Tejasvi Chakravarthy, Bharath Muthiah, and Sriram Sankar. 2021. Silent Data Corruptions at Scale. *arXiv preprint arXiv:2102.11245* (2021).
- [9] Shuguang Feng, Shantanu Gupta, Amin Ansari, and Scott Mahlke. 2010. Shoestring: probabilistic soft error reliability on the cheap. In *ACM SIGARCH Computer Architecture News*, Vol. 38. ACM, 385.
- [10] Giorgis Georgakoudis, Ignacio Laguna, Dimitrios S Nikolopoulos, and Martin Schulz. 2017. Refine: Realistic fault injection via compiler-based instrumentation for accuracy, portability and speed. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–14.
- [11] Olga Goloubeva, Maurizio Rebaudengo, M Sonza Reorda, and Massimo Violante. 2003. Soft-error detection using control flow assertions. In *Proceedings 18th IEEE Symposium on Defect and Fault Tolerance in VLSI Systems*. IEEE, 581–588.
- [12] Luanzheng Guo, Dong Li, and Ignacio Laguna. 2021. Paris: Predicting application resilience using machine learning. *J. Parallel and Distrib. Comput.* (2021).
- [13] Siva Kumar Sastry Hari, Sarita V Adve, and Helia Naeimi. 2012. Low-cost program-level detectors for reducing silent data corruptions. In *International Conference on Dependable Systems and Networks (DSN)*. IEEE, 1–12.
- [14] Siva Kumar Sastry Hari, Sarita V Adve, Helia Naeimi, and Pradeep Ramachandran. 2012. Relyzer: exploiting application-level fault equivalence to analyze application resiliency to transient faults. In *ACM SIGARCH Computer Architecture News*, Vol. 40. ACM, 123.
- [15] Peter H Hochschild, Paul Turner, Jeffrey C Mogul, Rama Govindaraju, Parthasarathy Ranganathan, David E Culler, and Amin Vahdat. 2021. Cores that don't count. In *Proceedings of the Workshop on Hot Topics in Operating Systems*. 9–16.
- [16] Yafan Huang, Shengjian Guo, Sheng Di, Guanpeng Li, and Franck Cappello. 2022. Mitigating silent data corruptions in hpc applications across multiple program inputs. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–14.
- [17] Charu Kalra, Fritz Previlon, Norm Rubin, and David Kaeli. 2020. ArmorAll: Compiler-based resilience targeting GPU applications. *ACM Transactions on Architecture and Code Optimization (TACO)* 17, 2 (2020), 1–24.
- [18] Ignacio Laguna, Martin Schulz, David F Richards, Jon Calhoun, and Luke Olson. 2016. Ipas: Intelligent protection against silent output corruption in scientific applications. In *Proceedings of the 2016 International Symposium on Code Generation and Optimization*. ACM, 227–238.
- [19] Sabuj Laskar, Md Hasanur Rahman, and Guanpeng Li. 2022. Tensorfi+: A scalable fault injection framework for modern deep learning neural networks. In *2022 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 246–251.
- [20] Sabuj Laskar, Md Hasanur Rahman, Bohan Zhang, and Guanpeng Li. 2022. Characterizing deep learning neural network failures between algorithmic inaccuracy and transient hardware faults. In *2022 IEEE 27th Pacific Rim International Symposium on Dependable Computing (PRDC)*. IEEE, 54–67.
- [21] Guanpeng Li and Karthik Pattabiraman. 2018. Modeling input-dependent error propagation in programs. In *2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 279–290.

- [22] Guanpeng Li, Karthik Pattabiraman, Chen-Yang Cher, and Pradip Bose. 2016. Understanding error propagation in GPGPU applications. In *International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 240–251.
- [23] Qining Lu, Karthik Pattabiraman, Meeta S Gupta, and Jude A Rivers. 2014. SDCTune: A model for predicting the SDC proneness of an application for configurable protection. In *Proceedings of the 2014 international conference on compilers, architecture and synthesis for embedded systems*. 1–10.
- [24] Robert E Lyons and Wouter Vanderkulk. 1962. The use of triple-modular redundancy to improve computer reliability. *IBM journal of research and development* 6, 2 (1962), 200–209.
- [25] Abdulrahman Mahmoud, Radha Venkatagiri, Khaliq Ahmed, Sasa Misailovic, Darko Marinov, Christopher W Fletcher, and Sarita V Adve. 2019. Minotaur: Adapting software testing techniques for hardware errors. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 1087–1103.
- [26] Nahmsuk Oh, Philip P Shirvani, and Edward J McCluskey. 2002. Control-flow checking by software signatures. *IEEE Transactions on Reliability* 51, 1 (2002), 111–122.
- [27] Nahmsuk Oh, Philip P Shirvani, and Edward J McCluskey. 2002. Error detection by duplicated instructions in super-scalar processors. *IEEE Transactions on Reliability* 51, 1 (2002), 63–75.
- [28] Lucas Palazzi, Guanpeng Li, Bo Fang, and Karthik Pattabiraman. 2019. A tale of two injectors: End-to-end comparison of ir-level and assembly-level fault injection. In *2019 IEEE 30th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 151–162.
- [29] Md Hasanur Rahman, Sheng Di, Shengjian Guo, Xiaoyi Lu, Guanpeng Li, and Franck Cappello. 2024. Druto: Upper-bounding silent data corruption vulnerability in gpu applications. In *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 582–594.
- [30] Md Hasanur Rahman, Sabuj Laskar, and Guanpeng Li. 2024. Investigating the impact of transient hardware faults on deep learning neural network inference. *Software Testing, Verification and Reliability* 34, 4 (2024), e1873.
- [31] Md Hasanur Rahman, Aabid Shamji, Shengjian Guo, and Guanpeng Li. 2021. Peppax: finding program test inputs to bound silent data corruption vulnerability in hpc applications. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–13.
- [32] George A Reis, Jonathan Chang, Neil Vachharajani, Ram Rangan, and David I August. 2005. SWIFT: Software implemented fault tolerance. In *International symposium on Code generation and optimization*. IEEE, 243–254.
- [33] Giacinto P Saggese, Nicholas J Wang, Zbigniew T Kalbarczyk, Sanjay J Patel, and Ravishankar K Iyer. 2005. An experimental study of soft errors in microprocessors. *IEEE micro* 25, 6 (2005), 30–39.
- [34] Philip P Shirvani, Nirmal Saxena, Nahmsuk Oh, Subhasish Mitra, Shu-Yi Yu, Wei-Je Huang, Santiago Fernandez-Gomez, Nur A Touba, and Edward J McCluskey. 1999. Fault-tolerance projects at stanford crc. In *MAPLD 1999-Annual Military and Aerospace Applications of Programmable Devices and Technologies Conference, 2 nd, Johns Hopkins Univ, APL*. Citeseer.
- [35] Alfred Spector and David Gifford. 1984. The space shuttle primary computer system. *Commun. ACM* 27, 9 (1984), 872–900.
- [36] Timothy Tsai, Nawanol Theera-Ampornpunt, and Saurabh Bagchi. 2012. A study of soft error consequences in hard disk drives. In *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN 2012)*. IEEE, 1–8.
- [37] Jiesheng Wei, Anna Thomas, Guanpeng Li, and Karthik Pattabiraman. 2014. Quantifying the accuracy of high-level fault injection techniques for hardware faults. In *44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 375–382.
- [38] Wujie Wen, Mengjie Mao, Xiaochun Zhu, Seung H Kang, Danghui Wang, and Yiran Chen. 2013. CD-ECC: Content-dependent error correction codes for combating asymmetric nonvolatile memory operation errors. In *2013 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 1–8.
- [39] Lishan Yang, Bin Nie, Adwait Jog, and Evgenia Smirni. 2020. Practical resilience analysis of GPGPU applications in the presence of single-and multi-bit faults. *IEEE Trans. Comput.* 70, 1 (2020), 30–44.
- [40] Lishan Yang, Bin Nie, Adwait Jog, and Evgenia Smirni. 2021. SUGAR: Speeding Up GPGPU Application Resilience Estimation with Input Sizing. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 5, 1 (2021), 1–29.
- [41] Li Yu, Dong Li, Sparsh Mittal, and Jeffrey S Vetter. 2014. Quantitatively modeling application resilience with the data vulnerability factor. In *SC'14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 695–706.
- [42] Ming Zhang and Naresh R Shanbhag. 2005. A CMOS design style for logic circuit hardening. In *2005 IEEE International Reliability Physics Symposium, 2005. Proceedings. 43rd Annual*. IEEE, 223–229.
- [43] Chijin Zhou, Mingzhe Wang, Jie Liang, Zhe Liu, and Yu Jiang. 2020. Zeror: Speed up fuzzing with coverage-sensitive tracing and scheduling. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. 858–870.

Appendix: Artifact Description

A Overview of Contributions and Artifacts

A.1 Paper’s Main Contributions

We propose PROTEGO– a lightweight input-aware SID protection technique that adapts protection strategies to runtime input of an application. The main contributions are summarized as follows.

- C₁** We perform an one-time fault injection-based vulnerability-guided offline search for exploring meaningful inputs that can maximally capture whole spectrum of SID protection variants.
- C₂** Based on our vulnerability-based search analysis on SID protection variance, we form a handful of input groups, where each group consists of inputs that share similar SID protection strategies. From each group, a representative SID protected executable is formed and saved for later use at runtime selection. This is also a one-time offline cost for each application.
- C₃** At runtime, given an online input, PROTEGO can efficiently selects an accurate protected executable from the set of representative executables from the offline phase. This selected protected executable can proxy the actual SID protected executable achievable under that online input. Our selection process is lightweight, which only depends on input-specific features, eliminating the need for costly fault injection or dynamic profiling.
- C₄** We evaluate PROTEGO using 10 widely used HPC applications in terms of SDC coverage loss mitigation, runtime execution efficiency. PROTEGO particularly offers significant advantage over existing state-of-the-art solutions when given online/runtime input is highly SDC-prone.

A.2 Computational Artifacts

A₁ <https://doi.org/10.5281/zenodo.16762873>

This study only has one artifact ID A_1 , which combines all the contributions C_1 , C_2 , C_3 and C_4 .

Artifact ID	Contributions Supported	Related Paper Elements
A_1	C_1	Section 4.2.1 Figure 5
A_1	C_2	Section 4.2.2 Figure 6
A_1	C_3	Section 4.2.3 & 4.2.4 Figure 7 & 8
A_1	C_4	Section 5 & 6 Figure 9, 10 & 11

B Artifact Identification

B.1 Computational Artifact A_1

Relation To Contributions

The artifact A_1 includes a source code for PROTEGO that offers the contributions C_1 to C_4 . For each of the 10 applications, the artifact

first performs a one-time vulnerability-guided search to determine the representative input groups followed by representative SID protected executable. At runtime, based our one-time analysis, the artifact projects an appropriate protected application executable from our developed efficient runtime SID decision mechanism given an online input.

Expected Results

For each application, the artifact will generate a SID-protected executable tailored to a given runtime input and user-specified protection level. This executable is expected to outperform existing state-of-the-art solutions in terms of both SDC coverage and runtime efficiency. For instance, consider a scenario where a 30% protection level yields 60% SDC coverage using a state-of-the-art approach. In such cases, our artifact A_1 is expected to offer SDC coverage higher than 60% if achievable, particularly for inputs that are highly SDC-prone. When higher coverage is not achievable for certain inputs due to already saturated coverage, A_1 would offer the comparable coverage as existing solutions offer, making the average loss of SDC coverage by A_1 reasonably lower than state-of-the-art solutions across runtime inputs.

Expected Reproduction Time (in Minutes)

Given an online input for one of 10 application, the runtime technique of the artifact may achieve $\sim 100\times$ deployment speedup over state-of-the-art solutions like full duplication (C_3 and C_4). Moreover, our one-time offline contributions – vulnerability-guided search (C_1) and representative SID protected executable formation (C_2) in the artifact – may take around 3500 minutes and 880 minutes respectively.

Artifact Setup (incl. Inputs)

Hardware. We conduct our experiments on a Linux server with Intel Xeon processor. This server is equipped with 24 processing cores and 512 GB of physical memory. Because fault injection experiments take a lot of time, we run them in parallel across 24 cores for both PROTEGO and baselines.

Software. We use a widely used fault injector LLFI for fault injection experiments. This tool is publicly available at the following URL: <https://github.com/DependableSystemsLab/LLFI>. Because of making applications compatible with LLFI, all of the 10 applications are compiled into LLVM IR when developing our code artifact. Moreover, as a standard practice in resilience community, we also implement the state-of-the-art solutions at LLVM IR level.

Datasets / Inputs. We evaluate the artifact using 10 widely used real-world HPC applications from 4 different benchmark suites, which covers different types of complex algorithms/computations. These applications are described in Table 1 of our paper.

Installation and Deployment. The artifact is implemented using LLVM IR, C and python language. For each application source code implemented in C, we first compile it to LLVM IR. The approach to convert an application benchmark into a single LLVM IR file is as

follows. We assume that the application has a few Cpp files (A.cpp, B.cpp and C.cpp) and a few C (D.c, E.c, F.c) files.

```
clang++ -emit-llvm -S A.cpp -o A.ll
clang++ -emit-llvm -S B.cpp -o B.ll
clang++ -emit-llvm -S C.cpp -o C.ll
clang -emit-llvm -S D.c -o D.ll
clang -emit-llvm -S E.c -o E.ll
clang -emit-llvm -S F.c -o F.ll
llvm-link A.ll B.ll C.ll D.ll E.ll F.ll -o application.ll
```

Moreover, fault injection is performed using widely used LLFI fault injector. Detailed instructions for the installations can be found at the following URL: <https://github.com/DependableSystemsLab/LLFI>. After the installation, 3 main executable commands such as *instrument*, *profile* and *injectfault* can be found under *bin* directory. A *input.yaml* can be found in *Utilities* folder of A_1 . This file controls how, when and where to inject fault injections. Before performing fault injections, user can modify this file according to their requirements. To perform one fault injection trial, the traditional usage is as follows. We here assume that the compiled application is *Pathfinder.ll*.

```
instrument --readable Pathfinder.ll -lm
profile llfi/Pathfinder-profiling.exe <Input>
injectfault llfi/Pathfinder-faultinjection.exe <Input>
```

Artifact Execution

This code artifact consists of 3 main tasks during one-time offline analysis: SDC vulnerability guided input search T_1 , input grouping formation T_2 , generate representative protected executable for each group T_3 . So, the workflow during offline analysis is $T_1 \rightarrow T_2 \rightarrow T_3$. At runtime, given a unseen input, the code artifact first extracts input argument-based features T_4 , then projects the input group based on the extracted features being fed to the trained MLP model T_5 , and finally selects an appropriate protected executable (from pre-generated executables during offline) based on input grouping projection T_6 . So, the workflow during online is $T_4 \rightarrow T_5 \rightarrow T_6$. Execution scripts are provided for each task in our publicly available github repository.

Artifact Analysis (incl. Outputs)

Given a runtime input for a LLVM-compiled application, the code artifact will efficiently provide an input-aware appropriate SID protected executable that would improve the SDC coverage and runtime efficiency compared to the existing state-of-the-art solutions. Given a runtime input, A_1 efficiently provides SID protected executables under 4 protection levels: 10%, 30%, 50% and 70%. User has the option to specify other protection levels.